

D ADDITIONAL EXPERIMENTS ON AR AND ATR

Many parameters of synthetic tasks like AR and ATR and the model architectures we tested have interesting effects on behavioural and mechanistic metrics, but not all experiments could fit in our main text. Therefore, we include additional interesting observations in this appendix.

D.1 ATTENTION NEEDS POSITION EMBEDDINGS

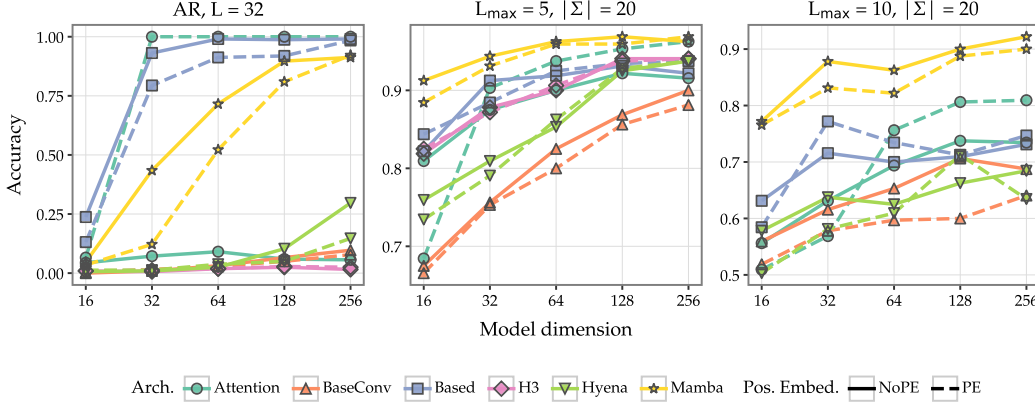


Figure 9: **Position embedding:** Model accuracy on AR and two ATR settings with and without absolute position embeddings.

Due to an initial configuration mistake, we accidentally trained all architectures with absolute position embeddings; in the zoology codebase (Arora et al., 2024a), only Attention is meant to be trained in this way. Fortuitously, this resulted in an interesting ablation: do SSMs, which are usually trained without it, also benefit from position embeddings?

Behavioural results. Our results in Figure 9 resoundingly show no: SSMs generally perform worse with position embeddings (PE). Attention is highly dependent on PE; performance on AR drops from 100.00% to 5.62% at $d = 256$ with NoPE. Attention lacks recurrence, unlike SSMs, so this is not surprising. However, on ATR, at smaller dimensionalities NoPE actually outperforms PE. Attention. Further ablations ought to consider alternative PE methods such as RoPE and Alibi.

D.2 MAMBA’S SOLUTION TO ATR DOES GENERALISE

We reuse the settings from our ATR experiment ($L = 5$, $|\Sigma| = 20$) and construct a new dataset with a train–test split on query–answer pairs. Specifically, 80% of possible unique query–answer pairs are provided in the training set, while 20% are only in the test set and thus never trained on. We seek to assess whether models learn a general mechanism for parent–child relations in ATR or if the impressive results of Mamba (as well as Attention and Based) are merely the result of better memorisation of the PCFG parameters. This setup is akin to Wang et al. (2024)’s technique of train–test split on multi-hop queries; we provide supervision on individual query and answer types, but not on some compositions of them.

Behavioural results. We select the checkpoint with the highest dev accuracy for each architectural and dimensionality setting, after sweeping LR. We plot the dev and test accuracies of each of these checkpoints in Figure 10a; all models have much lower test accuracy (e.g. Attention with $d = 256$ has 95.62% dev and 68.12% test accuracy). Attention achieves the greatest dev accuracies on $d \geq 32$. Mamba’s relative ranking is lower than on the in-distribution setting in §5.2, but it still achieves the overall second-highest dev accuracy (65.00% at $d = 128$). Surprisingly, H3 generalises well despite its poor dev accuracy, beating Mamba on test accuracy in 3 out of 5 settings.

We compare dev and test accuracies across all LRs in Figure 10b. We find that while Mamba does have unusually high dev accuracy given a selected test accuracy (indicating greater memorisation than models with other architectures), its dev accuracy is still generally higher than non-Attention

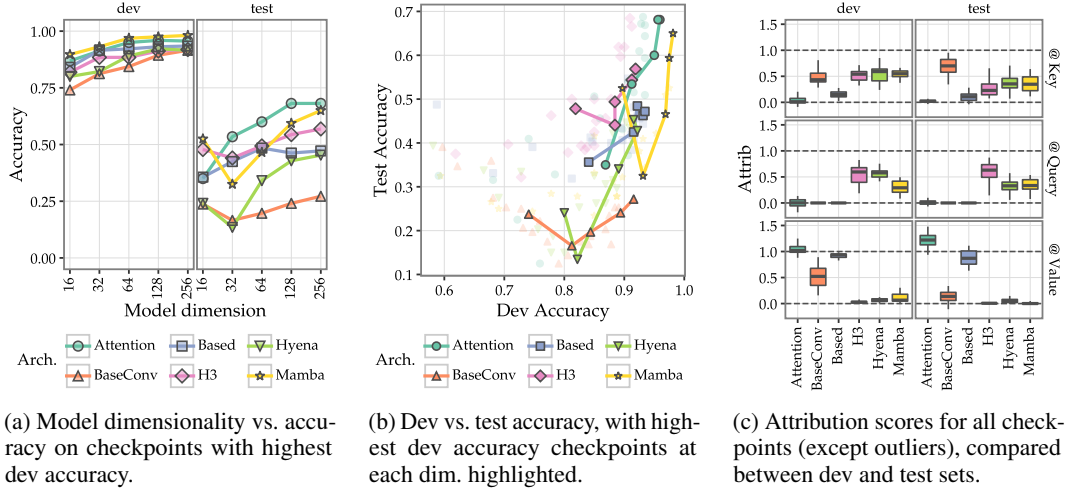


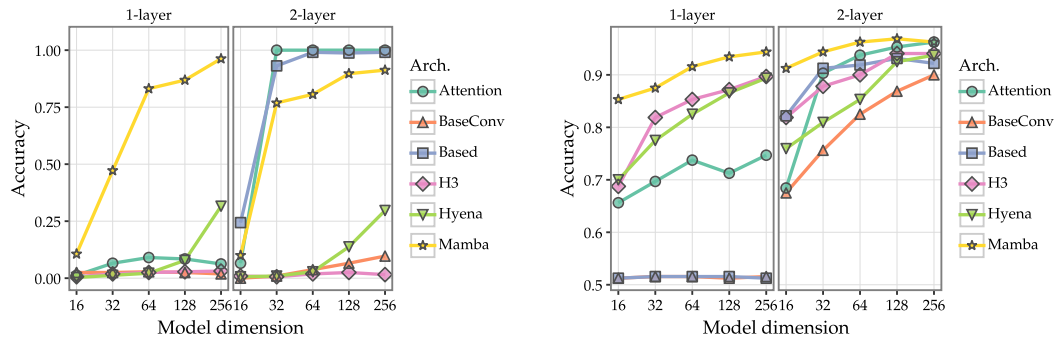
Figure 10: **Generalisation on Associative Treecall**: Accuracy and interchange intervention results on ATR with train–test split. Scores are reported on dev (with in-distribution query–answer pairs from training) and test (OOD). We highlight the checkpoint with the best dev score in each setting.

architectures. Interestingly, H3 has nearly Attention-level generalisation while BaseConv exhibits vanishingly little generalisation. Overall, behavioural metrics show that Mamba does nontrivially generalise on ATR, albeit not as well as Attention.

Mechanistic analysis. We report a summary of attribution scores at different tokens (*key*, *query*, *value*), comparing on dev and test sets across all checkpoints in Figure 10c. We find largely consistent mechanisms underlying behaviour on both dev and test, and these match attribution scores on ATR without train–test split. The only exception is that BasedConv does induction on the dev set but not nearly as much on the test set; its induction mechanism is more brittle than Attention and Based.

Overall, the induction mechanism is not more general than the direct retrieval mechanism; both Attention and Mamba show greater generalisation than other architectures despite their entirely different solutions, and our mechanistic evaluations confirm that this solution is consistent across in-distribution and out-of-distribution queries.

D.3 1-LAYER SSMS LEARN DIRECT RETRIEVAL ON AR AND ATR



(a) Accuracy of 1-layer vs. 2-layer models on AR, 32 key–value pairs. 1-layer induction models fail. (b) Accuracy of 1-layer vs. 2-layer models on ATR ($L = 5$, $|\Sigma| = 20$), with Based and BaseConv failing.

Figure 11: **1-layer models on AR and ATR**: Architectures that learn induction in the 2-layer setting fail to perform non-trivially with 1 layer. Mamba is highly performant with 1 layer on both tasks.

Throughout our experiments on AR and ATR, we have claimed that SSMS (except for Based and possibly BaseConv) learn a direct retrieval mechanism which does not require an intermediate step

like attention, i.e. only a single SSM layer is needed to learn AR and ATR. To verify this, we repeat AR and $L = 5$, $|\Sigma| = 20$ ATR experiments (without train–test split) with 1-layer models.

Behavioural results. We find comparable performance for direct retrieval models between 1-layer and 2-layer settings on AR (Figure 11a). In fact, at $d = 256$, 1-layer Mamba (96.25%) outperforms 2-layer Mamba (91.25%), as does Hyena (31.56% vs. 29.69%). 1-layer Based and BaseConv are architecturally identical, so we only report one; that architecture and Attention, both relying on induction in the 2-layer case, fail to learn AR with one layer. On ATR (Figure 11b), we see a more noticeable difference with layer count on all architectures, but again Attention, Based, and BaseConv become the worst architectures with one layer (e.g. 96.25% $\rightarrow$ 74.69% for Attention at $d = 256$).

D.4 SSMS PREFER LAYER 0 TO PERFORM AR

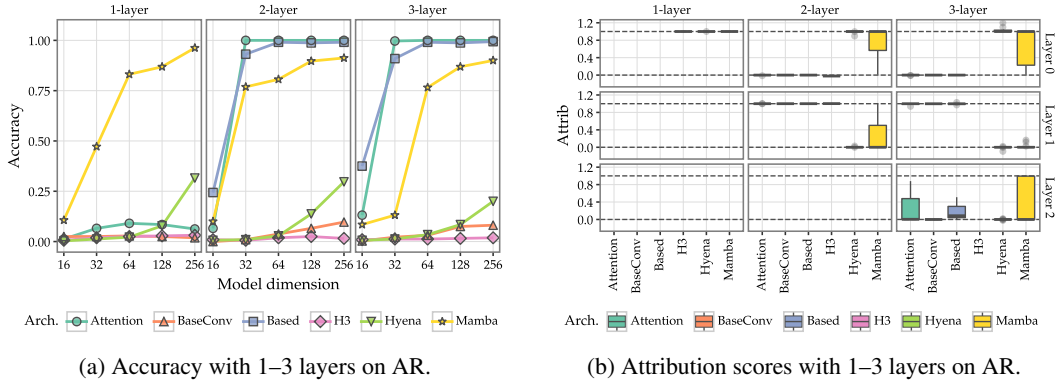


Figure 12: **Varying layer count on AR:** Behavioural and mechanistic evaluations for models with 1–3 layers on AR.

Since we have confirmed that the direct retrieval mechanism in SSMs requires only a single layer, we are curious which layer this mechanism forms in if more than two layers are present. We train models with up to three layers on AR and report results.

Behavioural results. 3-layer models perform about the same on AR as 2-layer models across architectures (Figure 12a), except for a large drop in performance for Mamba when $d = 32$; this may just be an optimisation failure.

Mechanistic analysis. For our mechanistic metric, instead of intervening on each block, we intervene at the sequence mixer’s output to the *query* token in each layer; this tells us if that layer is directly responsible for writing the answer to the output position. We apply the same filter as in §4.2, with a threshold of 0.01. Figure 12b shows that among performant models, Hyena and Mamba prefer layer 0 for performing AR no matter the layer count; however, some Mamba checkpoints learn the mechanism in the final layer as well (but never layer 1 in a 3-layer model). Attention, Based, and BaseConv prefer layer 1, which is expected since this is the second step of the induction mechanism. However, some checkpoints of Attention and Based also have non-zero attribution score at layer 2 in the 3-layer setting.

D.5 RIGHTMOST SIBLING QUERIES ARE TRIVIAL FOR ALL ARCHITECTURES

Since ATR has hierarchical structure, we attempted an initial experiment with multihop queries; specifically, we present queries where the answer is that terminal’s rightmost sibling terminal. Models are only trained on this type of query, not standard parent queries as reported in the main text. We train with the same settings in §4.1.

Behavioural results. In Figure 13 we show that all models (except Based and BasedConv with 1 layer, where they only have local convolutions) achieve greater than 80% accuracy at the task at all dimensionalities. We see slight improvement from 1-layer to 2-layer models but at this point performance is saturated and 3-layer does not help. Clearly, this task is extremely simple for all